

Lecture 1: Introduction to Programming Languages & C#

- [Lecture 1: Introduction to Programming Languages & C](#)
 - [1. What is a Programming Language?](#)
 - [2. Why Do We Use Programming Languages?](#)
 - [3. How Many Programming Languages Are There?](#)
 - [Categories of Programming Languages](#)
 - [Popular Programming Languages Today \(with 1-line definition & example\)](#)
 - [4. Difference Between Programming Languages \(Multi-Aspect Comparison\)](#)
 - [5. What is C#?](#)
 - [When Was It Developed & Who Developed It?](#)
 - [Why Was C# Developed?](#)
 - [6. Why Should We Learn C#?](#)
 - [7. What is C# Used For? \(Real-World Applications\)](#)
 - [8. Career & Job Aspects Related to C](#)
 - [Job Roles Where C# is Required](#)
 - [Companies That Use C# / .NET \(Real-World Examples\)](#)
 - [Salary & Demand Snapshot \(General Industry Trend\)](#)
 - [9. Quick Recap \(Key Takeaways\)](#)
 - [10. What's Next?](#)

Lecture 1: Introduction to Programming Languages & C

A complete beginner-to-career-ready guide. Read this once for understanding, and come back to it as a reference throughout your programming journey.

1. What is a Programming Language?

A **programming language** is a formal set of instructions (syntax + rules) that humans use to communicate commands to a computer. A computer only understands **binary (0 and 1)** — machine language. Since writing everything in binary is practically impossible for humans, programming languages act as a **bridge** between human logic and machine execution.

Simple definition: > A programming language is a structured way of writing instructions that a computer can understand (directly or after translation) to perform a specific task.

Analogy: Think of a programming language like a recipe book. The computer is the chef, and the code

is the recipe. If the recipe (code) is written clearly and correctly, the chef (computer) produces the exact dish (output) you wanted.

2. Why Do We Use Programming Languages?

We use programming languages because:

1. **To instruct computers** – Computers cannot think on their own; they need precise instructions.
2. **To automate tasks** – Repetitive tasks (payroll calculation, sending emails, sorting data) can be automated.
3. **To build software/applications** – Websites, mobile apps, games, operating systems, banking systems — all are built using programming languages.
4. **To solve real-world problems** – From Google Maps routing to Netflix recommendations, code solves logistical, mathematical, and business problems.
5. **To create economic value** – Software runs the modern economy (banking, healthcare, e-commerce, aviation, defense).

Real-world example: When you open Swiggy/Zomato and order food, dozens of programs run behind the scenes — one to track your location, one to calculate delivery charges, one to process payment, one to notify the restaurant. All are written using programming languages.

Basically as we talk with other persons in our language like hindi, english, bengali etc, we also talk with computer using programming language and make them perform task according to our given instructions.

3. How Many Programming Languages Are There?

There is no single fixed number — estimates suggest **over 700 actively used programming languages**, and thousands more exist historically or academically (some sources say 8,000+ if you count every dialect and esoteric language ever created). However, only about **50-60 languages** are commonly used in the industry today.

Categories of Programming Languages

Category	Description	Examples
Low-Level Languages	Close to machine code, hardware-specific, fast but hard to write	Machine Language, Assembly Language
High-Level Languages	Human-readable, portable across machines, needs translation	C, C++, Java, Python, C#
Procedural Languages	Code executes step-by-step using functions/procedures	C, Pascal, BASIC
Object-Oriented Languages (OOP)	Code organized around “objects” (data + behavior)	Java, C++, C#, Python
Functional Languages	Based on mathematical functions, avoids changing state	Haskell, Erlang, F#, Scala

Scripting Languages	Usually interpreted, used for automation, web	JavaScript, Python, PHP, Ruby
Markup/Styling Languages*	Not “programming” technically but structure content	HTML, CSS, XML

(HTML/CSS are markup languages, not programming languages, since they don’t perform logic/computation — a common interview question!)

Popular Programming Languages Today (with 1-line definition & example)

Language	Definition	Example Use
C	Procedural, low-level-ish language, foundation of modern languages	Operating Systems (Linux kernel)
C++	Object-oriented extension of C, high performance	Game engines (Unreal Engine)
Java	Platform-independent OOP language (“write once, run anywhere”)	Android apps, banking systems
Python	Simple, readable, general-purpose interpreted language	AI/ML, Data Science, automation
C#	Modern, OOP language by Microsoft, runs on .NET	Windows apps, games (Unity), enterprise software
JavaScript	Scripting language for making web pages interactive	Web front-end (React), Node.js backend
PHP	Server-side scripting language for web	WordPress, dynamic websites
Swift	Apple’s language for iOS development	iPhone/iPad apps
Kotlin	Modern language for Android, interoperable with Java	Android apps
Go (Golang)	Fast, simple language by Google for backend/cloud systems	Cloud infrastructure (Docker, Kubernetes)
Rust	Memory-safe, high-performance systems language	Browsers (Firefox), system tools
SQL	Domain-specific language for managing databases	Querying databases (MySQL, SQL Server)

4. Difference Between Programming Languages (Multi-Aspect Comparison)

--	--	--

Aspect	Low-Level (C, Assembly)	High-Level (Java, Python, C#)
Readability	Hard to read/write	Easy, human-friendly
Execution Speed	Very fast	Slightly slower (due to abstraction)
Portability	Machine-dependent	Mostly platform-independent
Memory Control	Manual (pointers)	Managed automatically (garbage collection)
Learning Curve	Steep	Easier for beginners
Use Case	OS, drivers, embedded systems	Apps, web, enterprise software

Aspect	Compiled Languages (C, C++, C#)	Interpreted Languages (Python, JavaScript)
How it runs	Translated fully to machine code before execution	Translated line-by-line at runtime
Speed	Faster execution	Generally slower
Error Detection	Errors caught before running (compile-time)	Errors found while running (runtime)
Example	C# compiles to IL (Intermediate Language), then runs via CLR (Just-In-Time)	Python code is interpreted by the Python interpreter

Aspect	Procedural (C)	Object-Oriented (C#, Java)	Functional (Haskell, F#)
Core Idea	Sequence of instructions/functions	Data + behavior bundled into "objects"	Functions as first-class entities, no side effects
Reusability	Limited	High (via inheritance, polymorphism)	High (via pure functions)
Real-world modeling	Difficult for complex systems	Excellent (models real entities like Car, Employee)	Great for mathematical/parallel computation

5. What is C#?

C# (pronounced "C-Sharp") is a modern, general-purpose, **object-oriented programming language** developed by **Microsoft**. It runs on the **.NET platform/framework** and is used to build a wide range of applications — from desktop software to web apps, games, and cloud services.

Formal Definition: > C# is a strongly-typed, multi-paradigm (object-oriented + component-oriented) programming language designed for building robust and scalable applications on the .NET ecosystem.

When Was It Developed & Who Developed It?

- **Developed by:** A team at **Microsoft**, led by **Anders Hejlsberg** (who earlier created **Turbo Pascal** and **Delphi**, and later led **TypeScript**).
- **Year:** Development began in **1999-2000**, and C# was officially released in **2002** alongside the **.NET Framework 1.0**.
- **Current status:** Now an open-source, cross-platform language maintained by Microsoft and the .NET Foundation (via **.NET Core / .NET 5+**), with the latest stable versions receiving yearly updates (e.g., C# 12 released alongside .NET 8).

Why Was C# Developed?

Microsoft created C# for a few key strategic and technical reasons:

1. **To compete with Java** – In the late 1990s, Microsoft’s relationship with Sun Microsystems (Java’s creator) broke down over licensing disputes. Microsoft wanted its own modern, Java-like language.
2. **To fix pain points of C++** – C++ was powerful but complex, error-prone (manual memory management, pointers). Microsoft wanted a language that combined **C++’s power** with the **simplicity of Visual Basic**.
3. **To unify Windows development** – Microsoft wanted a single, modern language for its new **.NET platform** that could build everything: desktop apps, web apps, and services.
4. **To provide automatic memory management** – Using a **garbage collector**, so developers don’t manually free memory (a huge source of bugs in C/C++).

In short: C# = “C++’s power + Java’s simplicity + Visual Basic’s ease + Microsoft’s ecosystem integration.”

6. Why Should We Learn C#?

1. **Beginner-Friendly yet Powerful** – Clean syntax, strong typing, great for learning core OOP concepts properly.
2. **Huge Industry Demand** – Used heavily in enterprise software, banking, and game development.
3. **Versatile (One language, many domains)** – Web (ASP.NET), Desktop (WPF/WinForms), Mobile (.NET MAUI), Games (Unity), Cloud (Azure), AI (ML.NET).
4. **Strong Microsoft & Community Backing** – Continuous updates, huge documentation, active community (Stack Overflow, GitHub).
5. **Great Tooling** – Visual Studio is one of the best IDEs in the world, with debugging, IntelliSense, and productivity features.
6. **Cross-Platform (Since .NET Core)** – C# code now runs on Windows, macOS, and Linux — no longer Windows-only.
7. **High-Paying Career Path** – C# developers, especially with Unity or Azure/.NET expertise, are well-paid globally.

7. What is C# Used For? (Real-World Applications)

Domain	Technology	Real-World Example
Desktop Applications	WPF, WinForms	Microsoft Office internal tools, financial trading desktop apps
Web Applications (Backend)	ASP.NET Core	StackOverflow’s backend is built

		on ASP.NET; many banking portals
Game Development	Unity Engine (uses C# as scripting language)	Games like <i>Pokémon GO</i> , <i>Hollow Knight</i> , <i>Cuphead</i> are built using Unity + C#
Mobile Apps	.NET MAUI / Xamarin	Cross-platform apps for Android & iOS from a single C# codebase
Cloud Computing	Microsoft Azure SDKs	Enterprises deploying scalable cloud services using C# and Azure Functions
Enterprise Software	.NET Framework/.NET Core	ERP systems, banking software, insurance claim systems
IoT & Embedded Systems	.NET nanoFramework	Smart home devices, sensor-based systems
Game Engines outside Unity	MonoGame	Indie 2D games

Real-world example: The popular game **“Cuphead”** was built using **Unity + C#**. Similarly, **Stack Overflow**, one of the largest developer Q&A platforms, runs its backend on **ASP.NET (C#)**.

8. Career & Job Aspects Related to C

Job Roles Where C# is Required

Job Role	What They Do	Where C# Fits
.NET Developer / Backend Developer	Build server-side logic, APIs, databases	Core C# + ASP.NET Core
Full Stack Developer (.NET stack)	Build both frontend (Blazor/React) and backend	C# for backend, sometimes Blazor for frontend too
Game Developer (Unity Developer)	Build 2D/3D games, gameplay logic	C# scripting in Unity
Software Engineer (Enterprise Applications)	Build large-scale business software	C#, .NET, SQL Server
Cloud Engineer / Azure Developer	Deploy and manage cloud-based apps	C# + Azure services
Mobile App Developer	Build cross-platform apps	C# with .NET MAUI/Xamarin
Automation/QA Engineer	Write automated test scripts	C# with Selenium/SpecFlow
Game Engine/Tools Programmer	Build internal tools for game studios	C# and C++ combined

Companies That Use C# / .NET (Real-World Examples)

- **Stack Overflow** – backend built on ASP.NET (C#)
- **Microsoft** (obviously) – Office, Azure, Visual Studio, Windows tools
- **Unity Technologies** – game engine used by thousands of studios worldwide
- **Alibaba, Dell, Accenture, GoDaddy** – use .NET/C# in enterprise systems
- **Game Studios:** Team Cherry (*Hollow Knight*), Studio MDHR (*Cuphead*), Niantic (*Pokémon GO* used Unity+C#)

Salary & Demand Snapshot (General Industry Trend)

- C#/.NET developers are consistently listed among the **top 10 most in-demand backend skills** globally (per Stack Overflow Developer Survey & Indeed/LinkedIn job trends).
- Unity + C# skills are highly sought after in the **gaming industry**, especially with the boom of **mobile & indie games**.
- Entry-level to senior .NET developer roles are available across **IT services, product companies, fintech, and gaming studios**.

(Note: exact salary figures vary by country, experience, and company — always check current job portals like LinkedIn, Glassdoor, or Naukri for up-to-date numbers in your region.)

9. Quick Recap (Key Takeaways)

- A **programming language** is how humans instruct computers to perform tasks.
- There are **hundreds of programming languages**, categorized as low-level/high-level, compiled/interpreted, procedural/OOP/functional.
- **C#** was developed by **Anders Hejlsberg at Microsoft**, released in **2002**, to combine the power of C++, simplicity of Java/VB, and integrate deeply with the **.NET platform**.
- C# is used in **web, desktop, mobile, cloud, and game development** — most notably powering the **Unity game engine**.
- Learning C# opens doors to careers as a **.NET Developer, Game Developer, Cloud Engineer, and Enterprise Software Engineer**, with strong industry demand and solid earning potential.

10. What's Next?

In the upcoming lectures, we will start with: 1. Setting up the C# development environment (Visual Studio / VS Code + .NET SDK) 2. Writing your first “Hello World” program 3. Understanding variables, data types, and basic syntax 4. Control flow (if-else, loops) 5. Introduction to Object-Oriented Programming in C#

Tip for students: Don't just read — type out every example yourself. Programming is a skill learned by *doing*, not just reading.